DIGITAL ELECTRONICS LAB (ESC392)

GENAI-ASSISTED DESIGN ASSIGNMENTS

Assignments 1–9 | Department of IT / CSE(IoT) / CSE(IoTCSBT)

Particular	Details
Course	Digital Electronics Lab (ESC392)
Assignment coverage	Part B: GenAI-assisted design assignments (1–5); Part C: Lab Assignments (6–9)
AI model used	ChatGPT (GPT-6)
Simulator	Logisim Evolution
Note	The separate AI-assisted SLM/Agentic-AI Project List (P1–P9) is excluded, as requested.

Prepared in the concise lab-report style of Lab Assignment 6, with prompts, design explanations, circuit illustrations, component lists and verification.

Assignment 1: AI-Assisted Boolean Expression Simplifier

Objective: To simplify a four-variable Boolean function using a Karnaugh map, obtain its minimized SOP form, and implement and verify the equivalent logic circuit.

GenAI Model Used: ChatGPT — GPT-6

Prompt: Simplify F(A,B,C,D) = Σm(0, 2, 8, 10, 12, 14) using a four-variable Karnaugh map. Show the Gray-code arrangement, identify valid groups, derive the minimized Sum of Products expression, implement the expression using NOT, AND and OR gates in Logisim Evolution, and verify it against the original minterms.

1. Given Description / Expression

F(A, B, C, D) = Σm(0, 2, 8, 10, 12, 14)

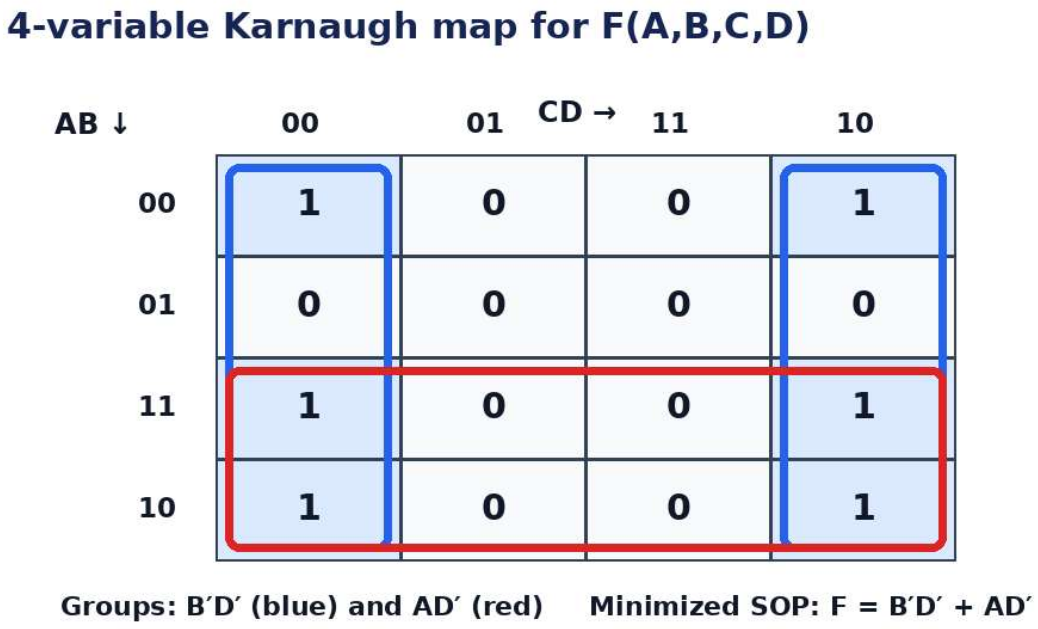


Figure 1.1: K-map with the two groups used to obtain the minimized SOP.

2. Design and Explanation

The six minterms are entered as 1s in a four-variable K-map. Use AB as the row variable pair and CD as the column pair, both in Gray-code order 00, 01, 11, 10.

Two overlapping groups of four give the implicants B′D′ and AD′. Therefore the minimized SOP expression is F = B′D′ + AD′ = D′(B′ + A).

3. Circuit Diagram / Implementation

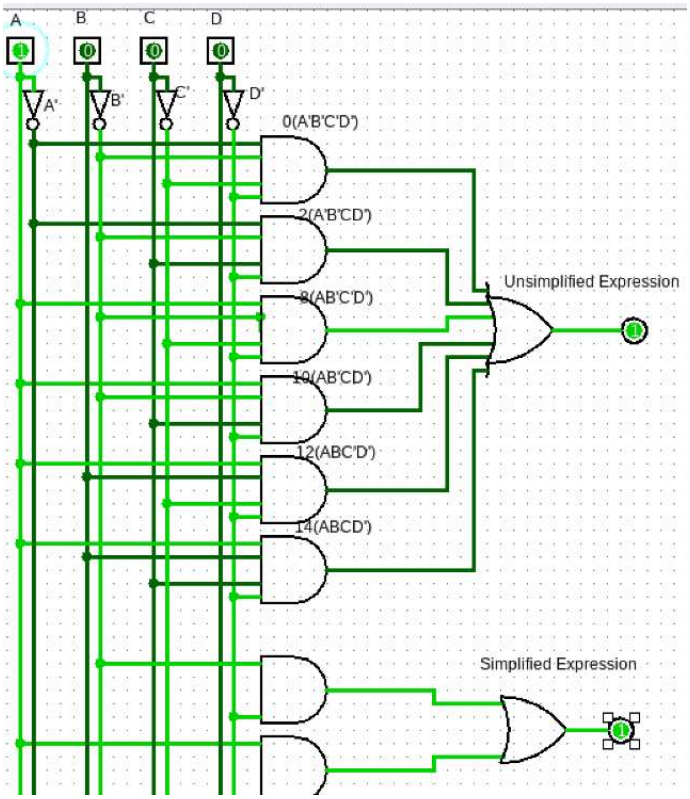


Figure 1.2: User-supplied Logisim Evolution circuit showing the original minterm network and minimized SOP implementation.

Logisim Evolution screenshot supplied for this assignment. The circuit and switch/output state shown are taken from the provided simulator image.

4. Components Used

- 2 NOT gates (to generate B′ and D′)
- 2 two-input AND gates
- 1 two-input OR gate
- Four input switches (A, B, C, D) and one output indicator F

5. Verification

A	B	C	D	F
0	0	0	0	1
0	0	0	1	0
0	0	1	0	1
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	1
1	0	0	1	0
1	0	1	0	1
1	0	1	1	0
1	1	0	0	1
1	1	0	1	0

A	B	C	D	F
1	1	1	0	1
1	1	1	1	0

6. Discussion / Result

The K-map reduces the original six-minterm function to F = B′D′ + AD′. The circuit uses fewer gates than a direct minterm implementation. The truth-table checks at the listed minterms and remaining input combinations confirm that the minimized expression preserves the original function.

Assignment 2: Natural Language to Logic Circuit Generator

Objective: To translate an English description of a logical condition into a Boolean expression, implement the corresponding gate-level circuit and verify its output.

GenAI Model Used: ChatGPT — GPT-6

Prompt: A warning light F turns ON when switch A is ON and at least one of sensors B or C is ON. Convert this statement into a Boolean expression, explain the required logic operations, draw the gate-level circuit, and verify all eight input combinations using a truth table.

1. Given Description / Expression

Warning light F is ON when A = 1 and (B = 1 or C = 1). Thus, F = A(B + C).

2. Design and Explanation

First combine B and C using a two-input OR gate. Its output is 1 when either sensor, or both sensors, are ON.

The OR output is then combined with A using a two-input AND gate. Therefore F becomes 1 only when A is ON and at least one of B or C is ON.

3. Circuit Diagram / Implementation

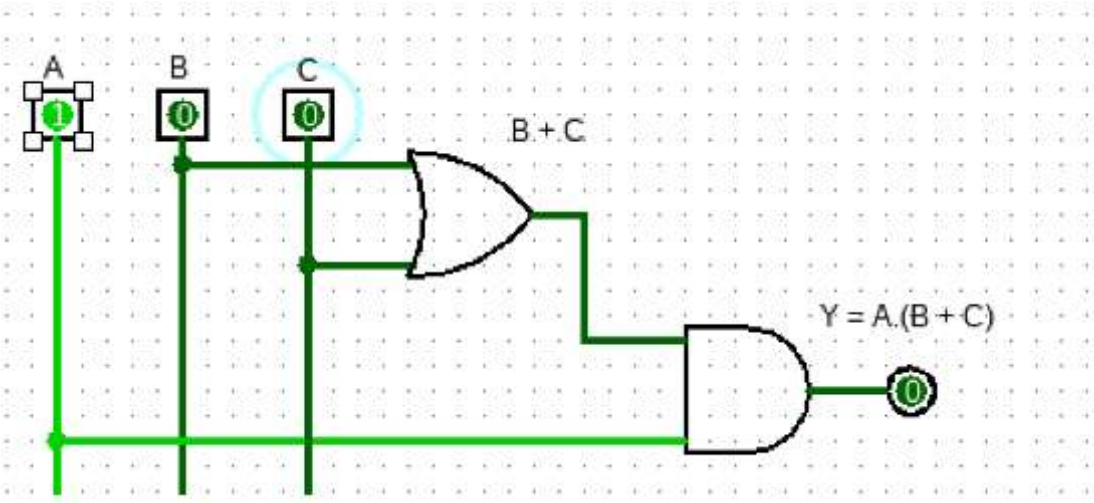


Figure 2.1: User-supplied Logisim Evolution implementation of F = A(B + C).

Logisim Evolution screenshot supplied for this assignment. The circuit and switch/output state shown are taken from the provided simulator image.

4. Components Used

- 1 × 2-input OR gate
- 1 × 2-input AND gate
- 3 input switches (A, B, C)
- 1 output indicator (F)

5. Verification

A	B	C	F (Expected)
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0

A	B	C	F (Expected)
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

6. Discussion / Result

The natural-language condition is represented by F = A(B + C). The truth table shows that the warning light turns ON only for the three combinations in which A = 1 and at least one sensor is active. The OR-then-AND implementation matches the expected behaviour.

Assignment 3: FSM Design for Traffic Light Controller

Objective: To model a traffic-light controller with a pedestrian-request input using a finite-state machine, derive its states and transitions, and implement the control logic using flip-flops and combinational logic.

GenAI Model Used: ChatGPT — GPT-6

Prompt: Design a finite-state machine for a road junction with RED, YELLOW and GREEN vehicle lights and a pedestrian request button. Define the states, inputs and outputs; describe safe state transitions including a pedestrian phase; provide a state table and a Logisim Evolution implementation plan using flip-flops and logic gates.

1. Given Description / Expression

Inputs: CLK, RESET, PED_REQ, TIMER_DONE. Outputs: vehicle RED/YELLOW/GREEN and pedestrian WALK/DON'T WALK.

2. Design and Explanation

Use four states: GREEN (normal vehicle flow), YELLOW (clearance interval), RED (vehicles stopped), and WALK (pedestrian crossing). The controller starts in GREEN after reset.

On timer completion, GREEN advances to YELLOW and YELLOW advances to RED. If a pedestrian request is pending, RED advances to WALK; otherwise it returns to GREEN after the safe red interval. WALK returns to GREEN when its timer completes.

Use two state flip-flops to encode four states. A timer-done input advances timed states; PED_REQ is sampled and retained until the pedestrian phase is served.

3. Circuit Diagram / Implementation

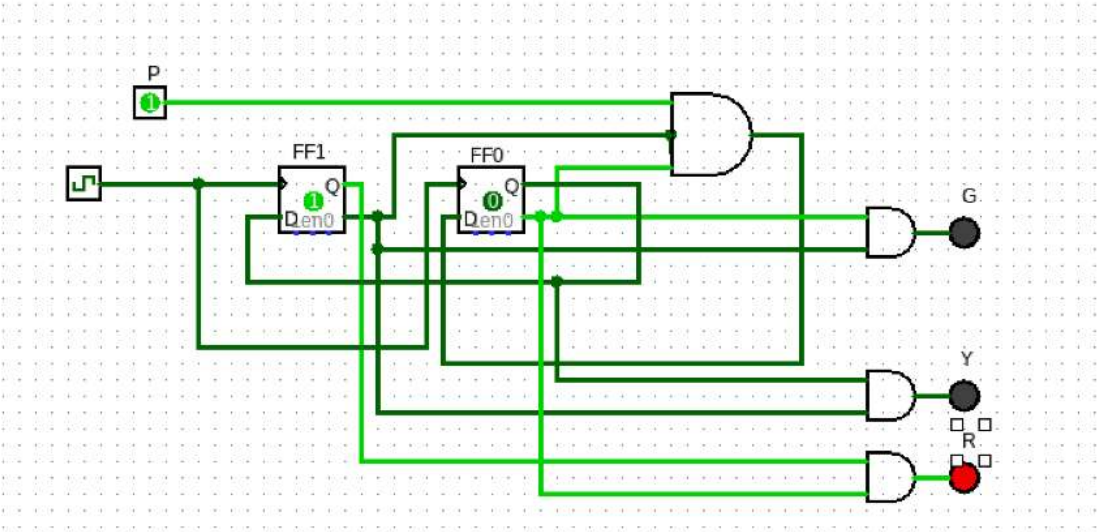


Figure 3.1: Logisim Evolution circuit screenshot with the RED vehicle-light output active.

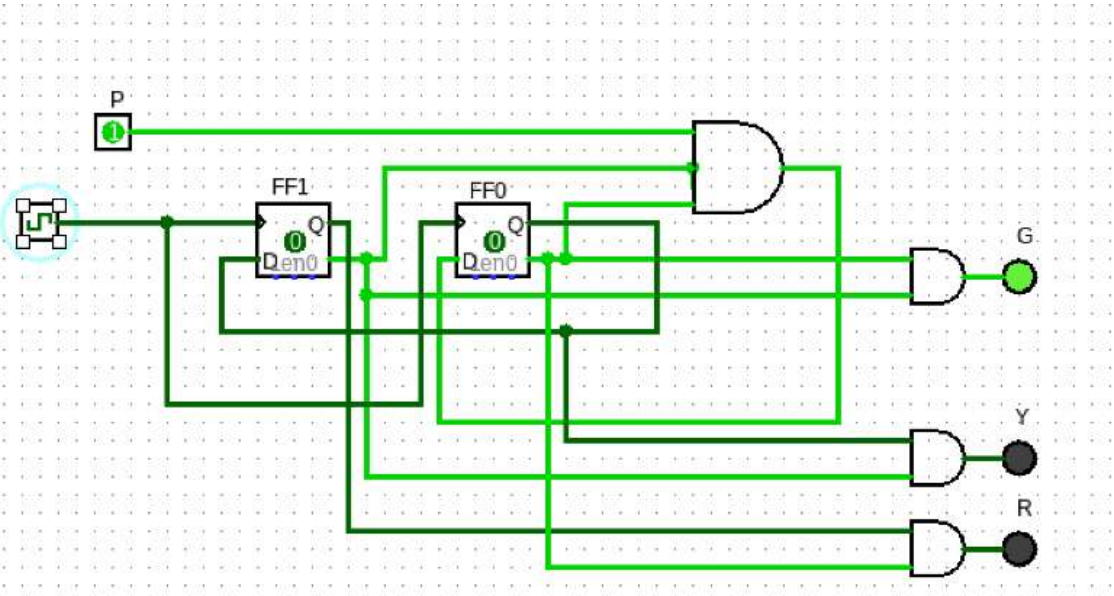


Figure 3.2: Logisim Evolution circuit screenshot with the GREEN vehicle-light output active.

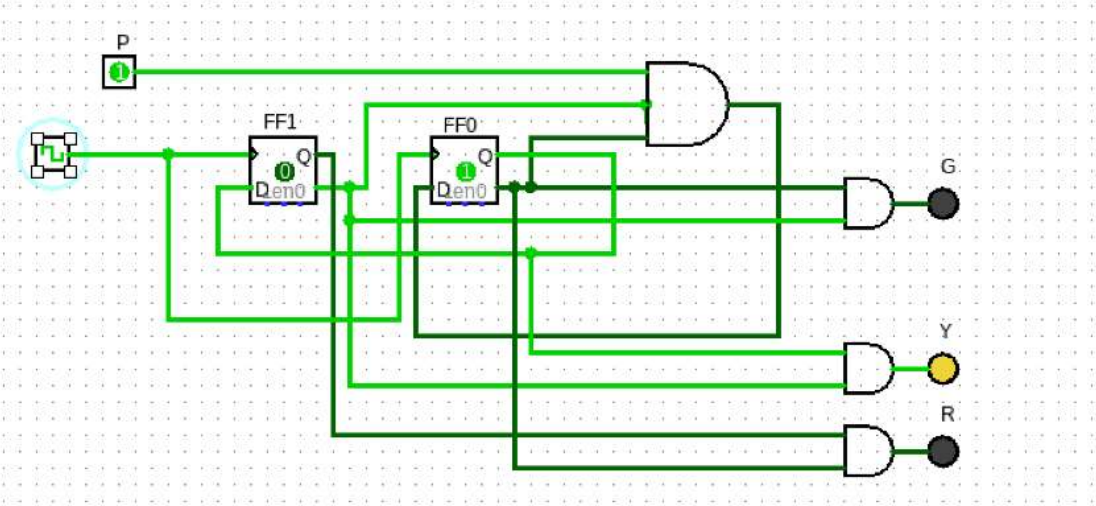


Figure 3.3: Logisim Evolution circuit screenshot with the YELLOW vehicle-light output active.

Logisim Evolution screenshot supplied for this assignment. The circuit and switch/output state shown are taken from the provided simulator image.

4. Components Used

- 2 D flip-flops for the four-state register
- Clock and reset inputs
- Pedestrian request input and timer-done input
- Combinational next-state logic (AND/OR/NOT gates)
- Output indicators for vehicle and pedestrian lamps

5. Verification

Current state	Condition	Next state	Vehicle output	Pedestrian output
GREEN	TIMER_DONE=0	GREEN	GREEN	DON'T WALK
GREEN	TIMER_DONE=1	YELLOW	GREEN	DON'T WALK
YELLOW	TIMER_DONE=1	RED	YELLOW	DON'T WALK

Current state	Condition	Next state	Vehicle output	Pedestrian output
RED	PED_REQ=1 and safe delay done	WALK	RED	WALK
RED	PED_REQ=0 and safe delay done	GREEN	RED	DON'T WALK
WALK	TIMER_DONE=1	GREEN	RED	DON'T WALK

6. Discussion / Result

The FSM separates normal vehicle movement, clearance, vehicle stop and pedestrian crossing into distinct states. The pedestrian request is served only after the vehicle signal reaches RED, preserving a safe transition. The state table can be translated into next-state logic and verified by stepping the clock in Logisim. The three simulator captures below show the red, green and yellow output states.

Assignment 4: AI-Based Fault Detection in Digital Circuits

Objective: To identify a deliberate logic error in a full-adder circuit by comparing its truth table with expected behaviour, correct the circuit, and verify the corrected design.

GenAI Model Used: ChatGPT — GPT-6

Prompt: Consider a full-adder circuit with inputs A, B and Cin and outputs SUM and Cout. Assume the implemented SUM path accidentally omits Cin. Use the full-adder truth table to identify the faulty rows, explain the error, derive the correct equations, and provide a corrected gate-level implementation for Logisim Evolution.

1. Given Description / Expression

Correct full-adder equations: SUM = A ⊕ B ⊕ Cin; Cout = AB + ACin + BCin. An equivalent carry equation is Cout = AB + Cin(A ⊕ B).

2. Design and Explanation

Calculate P = A ⊕ B using the first XOR gate. The second XOR gate combines P with Cin to produce SUM.

For carry, generate AB and Cin·P using AND gates, then OR the two terms. If the faulty circuit uses SUM = A ⊕ B only, rows with Cin = 1 expose the missing input path.

Compare the faulty output with the expected full-adder truth table. Any mismatch identifies the affected input combinations; restore the missing Cin connection to the second XOR gate.

3. Circuit Diagram / Implementation

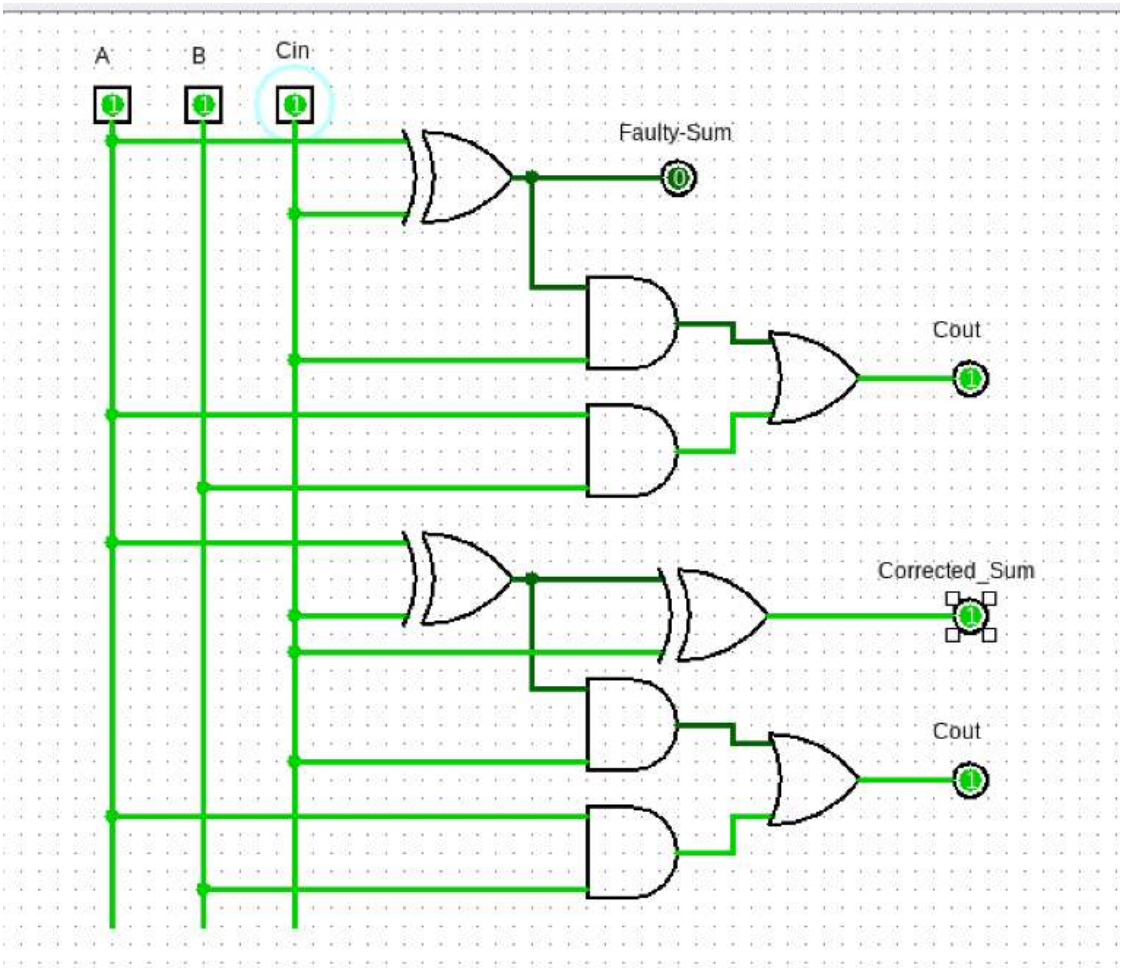


Figure 4.1: User-supplied Logisim Evolution full-adder circuit showing faulty and corrected SUM paths.

Logisim Evolution screenshot supplied for this assignment. The circuit and switch/output state shown are taken from the provided simulator image.

4. Components Used

- 2 XOR gates for SUM
- 2 AND gates and 1 OR gate for Cout
- 3 input switches (A, B, Cin)
- 2 output indicators (SUM, Cout)

5. Verification

A	**B**	**Cin**	Faulty SUM	Correct SUM	**Cout**
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	1	1	0
0	1	1	1	0	1
1	0	0	1	1	0
1	0	1	1	0	1
1	1	0	0	0	1
1	1	1	0	1	1

6. Discussion / Result

The fault is caused by omitting Cin from the SUM path. The correct SUM requires two cascaded XOR operations, while the carry output is generated by $AB + Cin(A \oplus B)$. Verification against all eight combinations confirms the corrected circuit.

Assignment 5: Digital Clock FSM Design with GenAI

Objective: To derive and implement a 24-hour digital clock using cascaded modulo counters, and verify the seconds, minutes and hours rollover behaviour.

GenAI Model Used: ChatGPT — GPT-6

Prompt: Design a digital clock that displays HH:MM:SS in 24-hour format. Explain the required modulo counters, show how carry signals cascade between seconds, minutes and hours, define the reset behaviour, and propose a Logisim Evolution circuit using counters and seven-segment or hex displays. Include representative rollover tests.

1. Given Description / Expression

Seconds: Mod-60; Minutes: Mod-60; Hours: Mod-24. Display range: 00:00:00 to 23:59:59.

2. Design and Explanation

A 1 Hz clock pulse increments the seconds counter. When seconds reach 59, the next pulse resets seconds to 00 and increments minutes.

When minutes reach 59 and seconds roll over, minutes reset to 00 and hours increment. When the hour count reaches 23 and the next minute rollover occurs, hours reset to 00.

Each Mod-60 counter can be built as cascaded Mod-10 and Mod-6 decimal digits. The hour counter must implement the 00–23 sequence, with reset after 23.

3. Circuit Diagram / Implementation

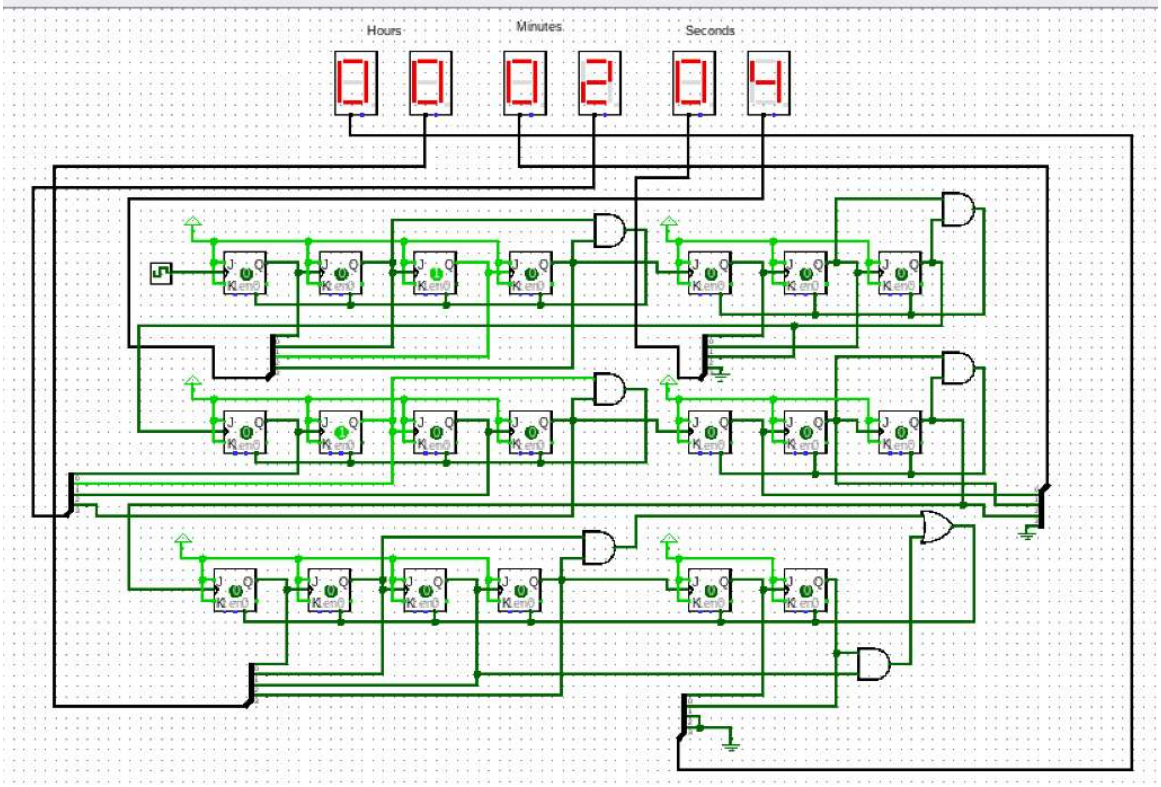


Figure 5.1: User-supplied Logisim Evolution 24-hour digital clock circuit with six seven-segment displays.

Logisim Evolution screenshot supplied for this assignment. The circuit and switch/output state shown are taken from the provided simulator image.

4. Components Used

- 1 Hz clock source
- Two Mod-60 counters (seconds and minutes)
- One Mod-24 counter (hours)
- Carry/enable connections between stages
- Reset button and display components

5. Verification

Test condition	Expected display	Action
Initial reset	00:00:00	All counters clear
One clock pulse	00:00:01	Seconds increment
After 59 seconds	00:01:00	Seconds roll over; minutes increment
At 01:59:59, next pulse	02:00:00	Minutes roll over
At 23:59:59, next pulse	00:00:00	24-hour rollover

6. Discussion / Result

The cascaded counter arrangement reproduces the timekeeping sequence. The seconds and minutes stages roll over at 60, while the hours stage rolls over at 24. Correct carry propagation and a common reset are essential for a valid 24-hour clock.

Assignment 6: Application Mapping of Sequential Building Blocks

Objective: To identify real-world applications of D flip-flops, Mod-N counters and SIPO/PISO shift registers, and demonstrate the Mod-60 counter in Logisim Evolution.

GenAI Model Used: ChatGPT — GPT-6

Prompt: For a D flip-flop, a Mod-N counter, and SIPO/PISO shift registers, identify one real-world system that uses each block and explain its role and working. Justify each application and describe a Logisim Evolution simulation for a Mod-60 counter that counts from 0 to 59 and resets to 0.

1. Given Description / Expression

Sequential blocks considered: D flip-flop; Mod-N counter; SIPO and PISO shift registers.

2. Design and Explanation

D flip-flop — input synchroniser: two D flip-flops in series synchronise an asynchronous push-button input to a microcontroller/FPGA clock. Each stage samples its D input on the rising clock edge.

Mod-N counter — digital clock: a Mod-60 counter counts seconds or minutes. Six flip-flops are sufficient because $2^6 = 64 \geq 60$.

SIPO/PISO — UART serial communication: PISO loads a parallel byte and shifts bits out serially; SIPO shifts incoming serial bits in and presents the received byte in parallel.

Mod-60 simulation: use a Mod-10 units counter and a Mod-6 tens counter. The units carry enables the tens counter; reset clears both counters.

3. Circuit Diagram / Implementation

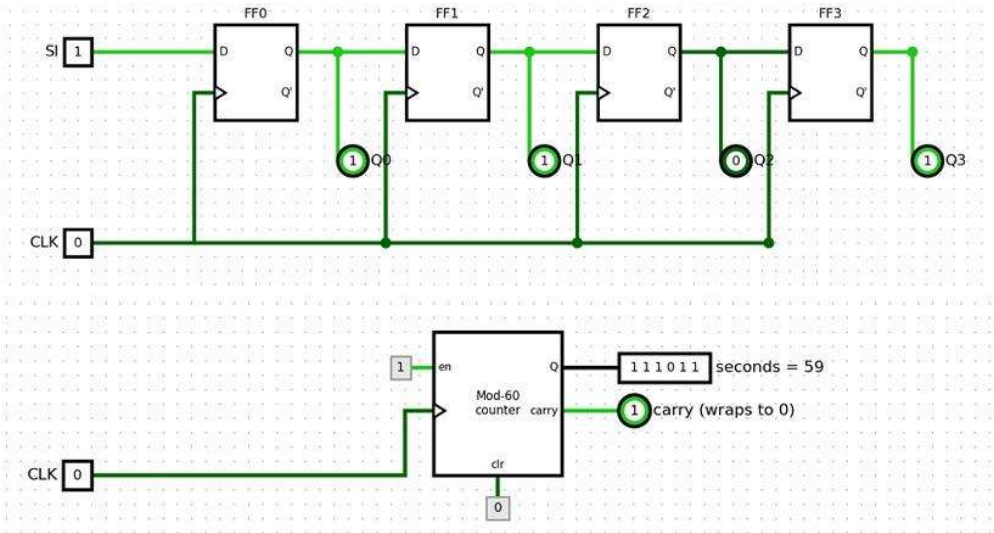


Figure 6.1: Logisim Evolution implementation showing a four-stage D flip-flop shift register and a Mod-60 counter.

4. Components Used

- D flip-flop(s) for synchronisation
- Two cascaded counters: Mod-10 and Mod-6
- Clock, enable and reset signals
- Two hex digit displays or decimal digit displays
- PISO and SIPO shift-register blocks for the UART illustration

5. Verification

Clock pulse	Tens digit	Units digit	Display	Note
0	0	0	00	Start
1	0	1	01	Units increments
9	0	9	09	Units carry asserted
10	1	0	10	Tens increments
59	5	9	59	Maximum count
60	0	0	00	Wrap-around
Any count + RESET	0	0	00	Immediate clear

6. Discussion / Result

The D flip-flop provides one-bit storage and clocked sampling. Mod-N counters are suitable for periodic sequences such as timekeeping, while shift registers perform serial/parallel conversion in communication links. The cascaded Mod-10 × Mod-6 counter implements the required 60-state sequence.

Assignment 7: Design a Mini Automation System — Automatic Parking Barrier

Objective: To apply combinational and sequential logic to an automatic parking barrier by defining inputs and outputs, deriving an FSM, and verifying its operation.

GenAI Model Used: ChatGPT — GPT-6

Prompt: Design an automatic parking barrier controller. Inputs include a valid-ticket signal, vehicle sensor, open/closed limit switches, obstacle sensor, clock and reset. Define the outputs for the barrier motor and status indicators. Derive a finite-state machine, explain transitions and safety handling, and outline a Logisim Evolution implementation and test cases.

1. Given Description / Expression

Inputs: VALID_TICKET, VEHICLE, OPEN_LIMIT, CLOSED_LIMIT, OBSTACLE, CLK, RESET. Outputs: MOTOR_UP, MOTOR_DOWN, GREEN_INDICATOR, RED_INDICATOR.

2. Design and Explanation

The controller uses CLOSED, OPENING, OPEN and CLOSING states. A valid ticket in CLOSED starts OPENING; OPEN_LIMIT changes the state to OPEN.

When the vehicle has passed, the controller enters CLOSING. CLOSED_LIMIT returns it to CLOSED. If OBSTACLE becomes active while closing, the controller immediately returns to OPENING to avoid closing on the obstacle.

State memory is implemented with D flip-flops. Next-state logic decodes sensor inputs and current state; output logic controls motor direction and indicators.

3. Circuit Diagram / Implementation

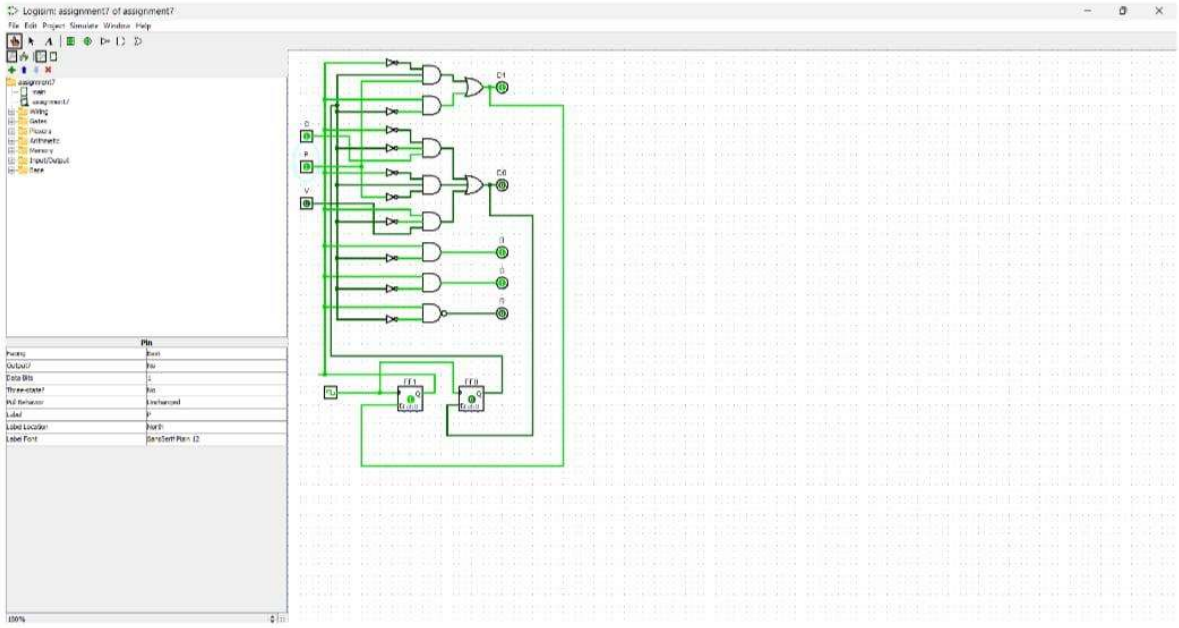


Figure 7.1: User-supplied Logisim Evolution circuit for the automatic parking barrier assignment.

4. Components Used

- 2 D flip-flops for four states
- AND, OR and NOT gates for next-state logic
- Ticket, vehicle, limit-switch and obstacle input switches

- Motor direction and status output LEDs
- Clock and reset

5. Verification

Current state	Input/event	Next state	Output
CLOSED	VALID_TICKET=1	OPENING	MOTOR_UP=1
OPENING	OPEN_LIMIT=1	OPEN	Motor OFF
OPEN	VEHICLE passed	CLOSING	MOTOR_DOWN=1
CLOSING	OBSTACLE=1	OPENING	MOTOR_UP=1
CLOSING	CLOSED_LIMIT=1	CLOSED	Motor OFF

6. Discussion / Result

The FSM provides a clear sequence for opening and closing the barrier. Limit switches stop the motor at the end positions, while the obstacle input overrides normal closing. The state-based design is easier to test and safer than a purely combinational controller.

Assignment 8: From Code to Circuit — 4-bit Mini-ALU and Program Counter

Objective: To implement a 4-bit arithmetic unit and a 4-bit program counter, then map arithmetic and loop execution in a short code fragment to hardware blocks.

GenAI Model Used: ChatGPT — GPT-6

Prompt: Build a 4-bit ALU supporting addition and subtraction using a mode-select input, and a 4-bit up-counter representing a program counter with enable and reset. Explain how c = a + b maps to registers and the ALU, and how a for-loop maps to repeated program-counter increments. Provide a labelled block diagram and verification cases.

1. Given Description / Expression

ALU mode: M=0 gives A+B; M=1 gives A−B. Program counter: PC ← PC+1 when ENABLE=1; RESET clears PC to 0000.

2. Design and Explanation

Store the two 4-bit operands in registers. Feed them to an adder/subtractor. For subtraction, invert B conditionally and set the carry-in to 1, implementing A + (B XOR M) + M.

The ALU result is stored in a result register. The program counter is a separate 4-bit register with an incrementer; it advances once per enabled instruction cycle.

For c = a + b, the registers hold a and b, the ALU performs addition, and the result register stores c. A loop is represented by repeated instruction fetch/execute cycles while the program counter advances.

3. Circuit Diagram / Implementation

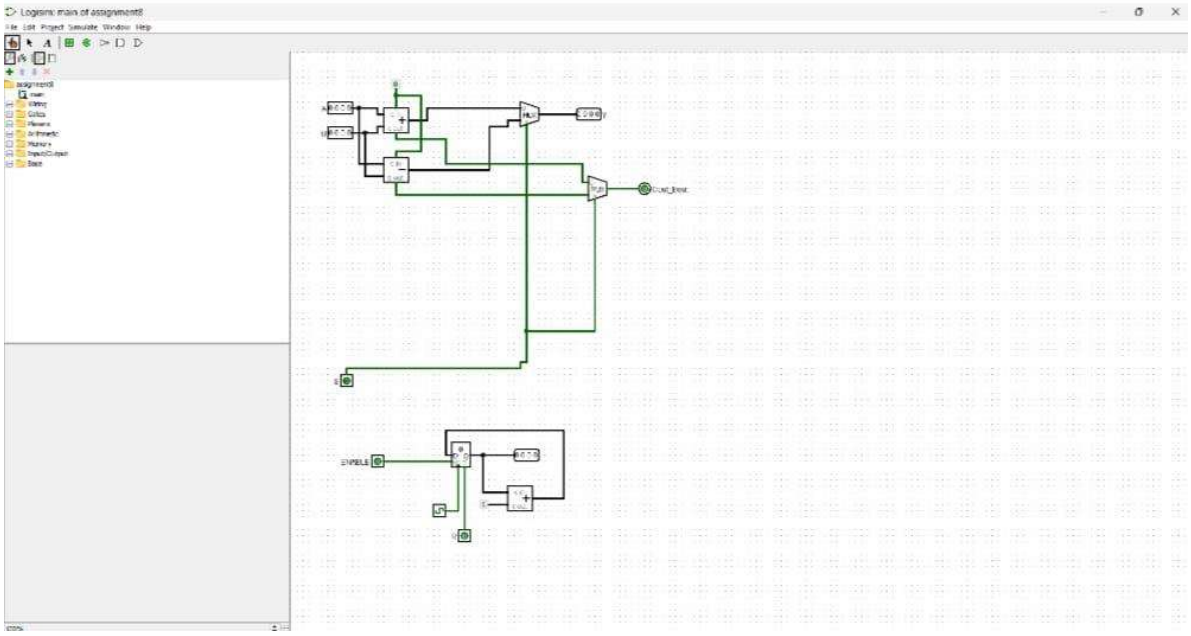


Figure 8.1: User-supplied Logisim Evolution circuit showing the 4-bit arithmetic unit and program-counter section.

4. Components Used

- Two 4-bit operand registers
- 4-bit adder/subtractor and mode-select control
- 4-bit result register
- 4-bit incrementer / up-counter for PC
- Enable and reset inputs

- LEDs or probes for operands, result and PC

5. Verification

Operation	A	B	Mode	Expected result
ADD	0011	0010	0	0101
ADD	1110	0001	0	1111
SUB	0110	0010	1	0100
SUB	0011	0101	1	1110 (two's-complement result)

6. Discussion / Result

The mini-ALU performs arithmetic selected by the mode input, while the program counter models instruction sequencing. Registers store operands and results between clock edges. This demonstrates how software operations are realized by datapath and control hardware.

Assignment 9: AI-Assisted Code-to-Gate Explainer

Objective: To use GenAI to map a small code condition to digital building blocks, simulate the proposed hardware, and verify that its outputs match the code's behaviour.

GenAI Model Used: ChatGPT — GPT-6

Prompt: Explain the hardware realization of the code if (a > b) c = a; else c = b; for two 4-bit unsigned inputs a and b. Identify the required comparator and multiplexer, draw the block-level circuit, and verify that c equals the larger input for representative and boundary cases. Mention any limitations of the hardware interpretation.

1. Given Description / Expression

GT = (a > b); c = GT ? a : b. The comparator output GT controls a 2:1, 4-bit-wide multiplexer.

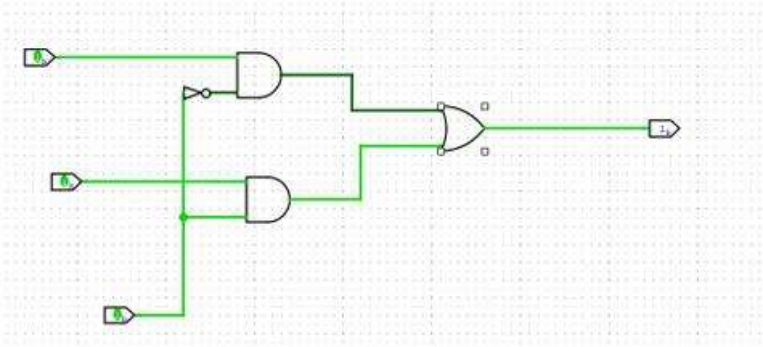
2. Design and Explanation

The magnitude comparator evaluates the two 4-bit operands and produces GT=1 when a is greater than b, otherwise GT=0.

The comparator output is connected to the select input of a 4-bit 2:1 multiplexer. When GT=1, the mux forwards a; when GT=0, it forwards b.

This is a combinational circuit: the output changes when either operand changes and does not require a clock. For equal operands, either input produces the same value.

3. Circuit Diagram / Implementation



4. Components Used

- One 4-bit magnitude comparator (or equivalent comparator logic)
- One 4-bit-wide 2:1 multiplexer
- Two 4-bit input switch groups (a and b)
- Four output indicators/probes for c

5. Verification

a	b	GT (a>b)	c output
0011	0010	1	0011
0010	0011	0	0011
0101	0101	0	0101
1111	0000	1	1111
0000	1111	0	1111

6. Discussion / Result

The code condition maps directly to a magnitude comparator followed by a multiplexer. The comparator decides which operand is larger, and the mux selects that operand. Verification confirms that the output follows the code for greater-than, less-than and equality cases.

GIT*HUB*_LINK - rickoche/Digital_Electronics: all 9 assingment of digital_electronics lab.